

Capstone Review Questions - Conversational Answers

Natural spoken-style rehearsal version

1. What problem does your project solve?

Answer: Honestly, we built this because teams lose track of work fast when everything is in different places. Our app keeps projects, tasks, owners, and status in one system so people can see what's going on without guessing.

2. What were the main objectives of your project?

Answer: Main goal was a real full-stack product, not just a toy demo. We wanted secure login, role-based access, real CRUD workflows, reporting on the dashboard, and a setup we could reliably run and test.

3. Explain your application's architecture.

Answer: It's a standard three-layer setup: React frontend, Express/Node backend, and MongoDB database. The frontend never talks directly to the database—it always goes through the API layer.

4. Walk us through a request from React to MongoDB and back.

Answer: User clicks something in React, that triggers an API call, backend middleware checks auth and validates input, then Mongoose writes/reads MongoDB. The API sends JSON back and React updates the screen.

5. What is the role of React in your application?

Answer: React is our UI engine. It handles routing, forms, table views, and state changes so users can actually interact with the backend cleanly.

6. Why did you use Node.js and Express?

Answer: We used Node/Express because it fits well with a TypeScript-first stack and lets us move quickly with route and middleware patterns. It also made auth, validation, and API organization straightforward.

7. What is MongoDB's role?

Answer: MongoDB is where our persistent data lives. Users, projects, tasks, and audit records all get stored there.

8. What happens when a user logs in?

Answer: Frontend posts credentials to the login endpoint. Backend verifies the password, returns a JWT plus user info, and then the frontend includes that token on protected requests.

9. What is JWT and why did you use it?

Answer: JWT is our auth token format. It's simple, stateless, and works well for protecting APIs and enforcing role-based access.

10. What happens if an unauthenticated user calls a protected API?

Answer: The request gets rejected by auth middleware before business logic runs. They get a 401 response.

11. What middleware does your application use?

Answer: We use middleware for JSON parsing, CORS, auth, role checks, request validation, error handling, and audit tracking.

12. Explain one CRUD operation end to end.

Answer: For task creation: user fills form in React, frontend sends POST to API, backend validates and saves task to MongoDB, then returns the created task and the UI updates.

13. What HTTP methods did you use?

Answer: GET for reads, POST for create, PUT/PATCH for updates, DELETE for removals—normal REST usage.

14. What HTTP status codes should your application return?

Answer: Typical ones: 200/201/204 for success, 400 for bad input, 401 for auth issues, 403 for forbidden access, 404 for missing resources, and 500 for unexpected server errors.

15. How did you validate user input?

Answer: We validate on the backend so rules can't be bypassed. Frontend validation is there too, mostly to improve user experience.

16. How do you handle errors?

Answer: Backend returns consistent error responses and status codes, and frontend catches failures and shows user-friendly messages instead of breaking silently.

17. How did you test your APIs?

Answer: We used automated integration/unit tests and also manual endpoint checks. Then we validated those same flows through the UI to make sure it all works end-to-end.

18. How can you prove that data is actually being stored in MongoDB?

Answer: In demo, we can create/update records in the app and then show the matching Mongo documents directly.

19. How did Git help your team?

Answer: Git gave us clean branch-based development, reviewable commits, and safe merging. It was essential once we had overlapping feature work.

20. What does npm do in your project?

Answer: npm handles dependencies and scripts for running, building, testing, and seeding both frontend and backend.

21. What was one significant technical challenge?

Answer: Big one was resolving merge conflicts between admin and audit features without breaking tests. We fixed conflicts, re-ran tests, and validated runtime behavior before pushing.

22. How did your team troubleshoot problems?

Answer: We debugged one layer at a time: UI behavior, network request/response, backend logs/middleware, then database state. That made troubleshooting way faster.

23. What did you know before the project compared with what you know now?

Answer: Before this, I understood pieces individually. Now I can build and explain how all layers work together under real change and merge pressure.

24. What is the most important thing you learned from the project?

Answer: Shipping reliable software is more than writing features. Good structure, testing, and repeatable environment workflows matter just as much.

25. What part of the project best demonstrates that you applied what you learned?

Answer: The admin + audit work is the clearest proof. It touches frontend, backend, auth, data modeling, middleware, tests, and deployment workflow.

26. Why should we believe this is more than just code that happens to work?

Answer: Because we can explain design choices, trace requests end-to-end, show test evidence, and reproduce setup/results consistently.

27. If you had another week, what would you improve?

Answer: I'd add stronger regression automation, better observability, and more polish on audit

insights/UI usability.

28. What would happen if MongoDB became unavailable?

Answer: Database-backed operations would fail, but the API should fail gracefully with controlled errors and useful logs/health signals.

29. Why shouldn't React communicate directly with MongoDB?

Answer: Because the backend is our security and business-rules boundary. Direct browser-to-DB access would bypass auth, validation, and authorization controls.

30. Can you explain your entire project in 30 seconds?

Answer: We built a full-stack task tracker with React, Express, and MongoDB, added JWT auth with role-based access, admin management, and audit logging, and validated it with tests plus containerized dev/prod workflows.

31. What did we build?

Answer: A role-aware task and project tracking platform with admin controls and traceable activity logging.

32. How does it work?

Answer: Users log in, use the React app, and all actions go through secured APIs to MongoDB. Permissions and audit behavior are enforced server-side.

33. What did I personally contribute?

Answer: I worked across backend routes/middleware, conflict resolution, audit integration, and test/rebuild/reseed validation to keep the branch stable.

34. What problem did we encounter and solve?

Answer: We had branch integration conflicts and a test regression after merging. I resolved both, revalidated tests, and confirmed clean mergeability into main.

35. What can I do now that I could not do before this training/project?

Answer: Now I can take a real feature from design through implementation, testing, merge, deployment workflow, and explain every part with confidence.